EXP 1:

A PYTHON PROGRAM TO IMPLEMENT UNIVARIATE, BIVARIATE AND MULTIVARIATE REGRESION

PROGRAM:

```
import pandas as pd
import
matplotlib.pyplot
as plt import
seaborn as sns
import numpy as np
df =
pd.read_csv('../input/iris-dataset/iris.csv'
) df.head(150)
df.shape
```

(150,5)

#univariate for sepal width

```
df.loc[df['variety']=='Setosa']
df_Setosa=df.loc[df['variety']=='Setosa']
df_Virginica=df.loc[df['variety']=='Virginica']
df_Versicolor=df.loc[df['variety']=='Versicolor']
plt.scatter(df_Setosa['sepal.width'],np.zeros_like(df_Setosa['sepal.width']))
plt.scatter(df_Virginica['sepal.width'],np.zeros_like(df_Virginica['sepal.width']))
plt.scatter(df_Versicolor['sepal.width'],np.zeros_like(df_Versicolor['sepal.width']))
plt.xlabel('sepal.width')
plt.show()
```

#univariate for sepal length

```
df.loc[df['variety']=='Setosa']
df_Setosa=df.loc[df['variety']=='Setosa']
df_Virginica=df.loc[df['variety']=='Virginica']
df_Versicolor=df.loc[df['variety']=='Versicolor']
plt.scatter(df_Setosa['sepal.length'],np.zeros_like(df_Setosa['sepal.length']))
plt.scatter(df_Virginica['sepal.length'],np.zeros_like(df_Virginica['sepal.length']))
plt.scatter(df_Versicolor['sepal.length'],np.zeros_like(df_Versicolor['sepal.length']))
plt.xlabel('sepal.length')
plt.show()
```

#univariate for petal width

```
df.loc[df['variety']=='Setosa']
df_Setosa=df.loc[df['variety']=='Setosa']
df_Virginica=df.loc[df['variety']=='Virginica']
df_Versicolor=df.loc[df['variety']=='Versicolor']
plt.scatter(df_Setosa['petal.width'],np.zeros_like(df_Setosa['petal.width']))
plt.scatter(df_Virginica['petal.width'],np.zeros_like(df_Virginica['petal.width']))
plt.scatter(df_Versicolor['petal.width'],np.zeros_like(df_Versicolor['petal.width']))
plt.xlabel('petal.width')
plt.show()
```

#univariate for petal length

```
df.loc[df['variety']=='Setosa']
df_Setosa=df.loc[df['variety']=='Setosa']
df_Virginica=df.loc[df['variety']=='Virginica']
df_Versicolor=df.loc[df['variety']=='Versicolor']
plt.scatter(df_Setosa['petal.length'],np.zeros_like(df_Setosa['petal.length']))
plt.scatter(df_Virginica['petal.length'],np.zeros_like(df_Virginica['petal.length']))
plt.scatter(df_Versicolor['petal.length'],np.zeros_like(df_Versicolor['petal.length']))
plt.xlabel('petal.length')
plt.show()
```

#bivariate sepal.width vs petal.width

```
sns.FacetGrid(df,hue='variety',size=5).map(plt.scatter,"sepal.width","petal.width").add_legend();
```

#bivariate sepal.length vs petal.length

```
sns.FacetGrid(df,hue='variety',size=5).map(plt.scatter,"sepal.length","petal.length").add_legend();
plt.show()
```

#multivariate all the features

```
sns.pairplot(df,hue="variety",size=2)
```

Ex. No. : 02 **Date:**

Register No : 231701046 **Name: SAM SUNDAR**

A python program to implement Simple linear regression using Least Square Method

AIM:

To implement Simple Linear Regression using the Least Square Method in Python.

REQUIREMENTS:
Jupyter Notebook with Python.

PROGRAM:

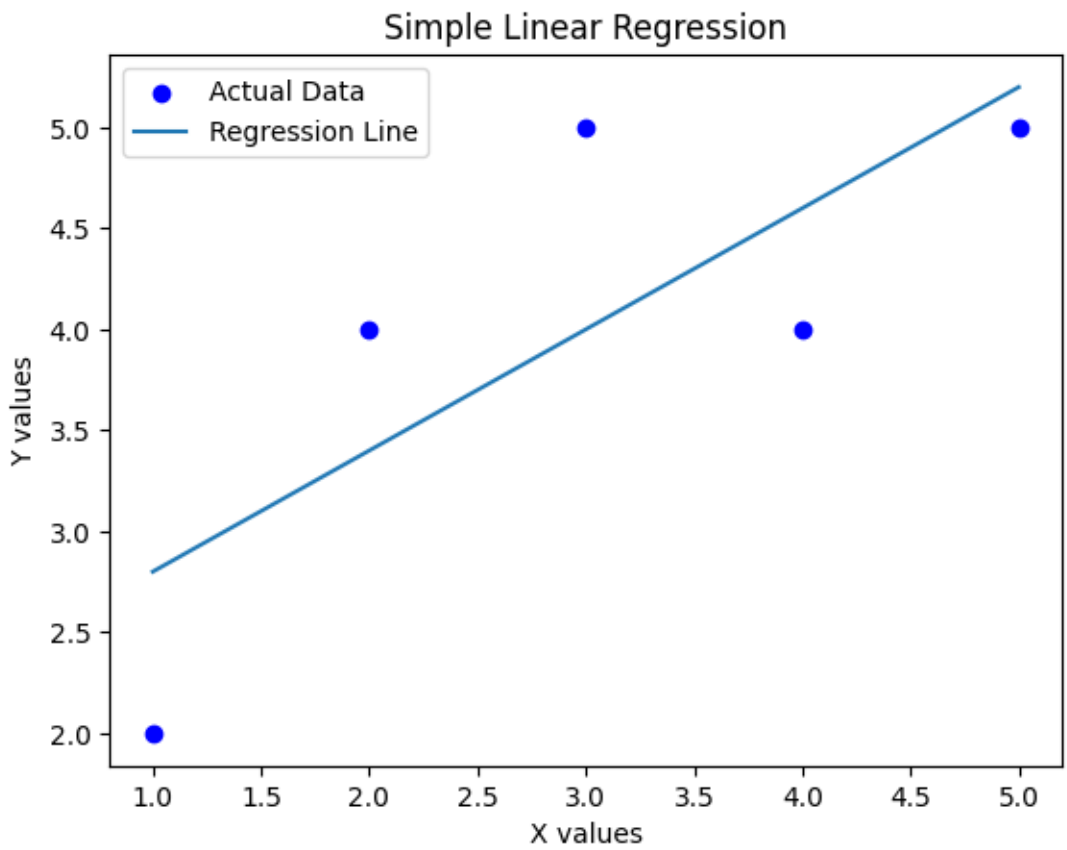
```
# Simple Linear Regression using Least Squares Method

import numpy as np

import matplotlib.pyplot as plt

# Input data

x = np.array([1, 2, 3, 4, 5])

y = np.array([2, 4, 5, 4, 5])

n = len(x)

# Calculate sums

sum_x = np.sum(x)

sum_y = np.sum(y)

sum_xy = np.sum(x * y)

sum_x2 = np.sum(x * x)

# Calculate slope (b1) and intercept (b0)
```

```
b1 = (n * sum_xy - sum_x * sum_y) / (n * sum_x2 - sum_x**2)
b0 = (sum_y - b1 * sum_x) / n

print("Slope (b1):", b1)
print("Intercept (b0):", b0)

# Predict values
y_pred = b0 + b1 * x
# Plot graph
plt.scatter(x, y)
plt.plot(x, y_pred)
plt.xlabel("X values")
plt.ylabel("Y values")
plt.title("Simple Linear Regression")
plt.show()
```

OUTPUT:

Slope (b1): 0.6

Intercept (b0): 2.2



RESULT:

Thus, the Simple Linear Regression model was successfully implemented using the Least Squares Method.

Ex. No. : 03 **Date:**

Register No : 231701046 **Name: SAM SUNDAR**

A python program to implement logistic model.

AIM:

To implement the Logistic Model (Logistic Regression) using Python.

REQUIREMENTS:

Jupyter Notebook with Python.

PROGRAM:

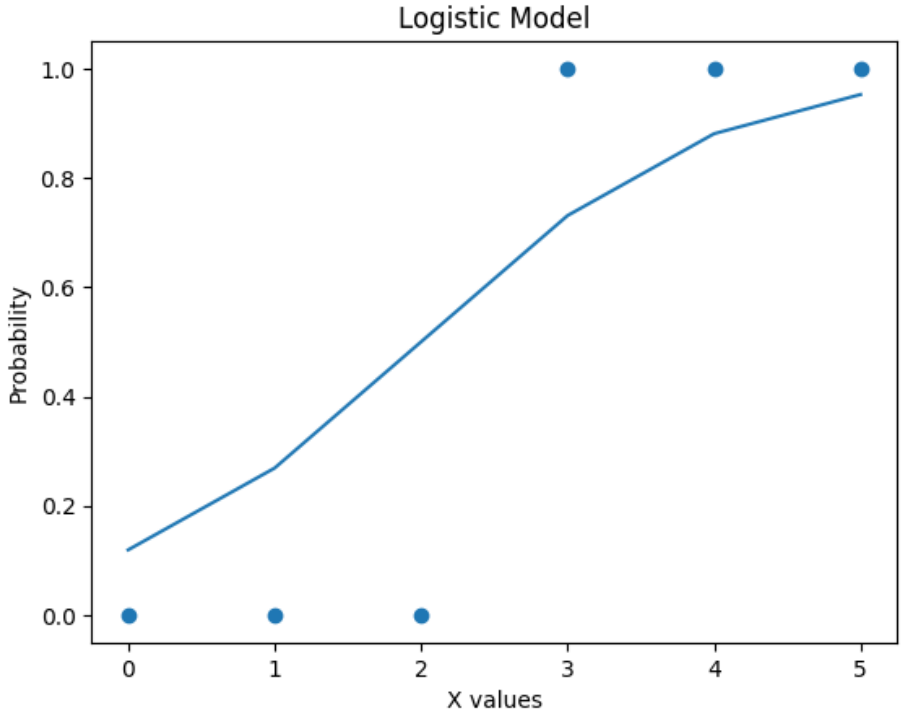
```
# Logistic Model Implementation

import numpy as np

import matplotlib.pyplot as plt

x = np.array([0, 1, 2, 3, 4, 5])

y = np.array([0, 0, 0, 1, 1, 1])

b0 = -2   # Intercept

b1 = 1    # Slope

def sigmoid(z):

    return 1 / (1 + np.exp(-z))

z = b0 + b1 * x

y_pred = sigmoid(z)


print("Predicted Probabilities:")

print(y_pred)

# Plot graph
```

```
plt.scatter(x, y)

plt.plot(x, y_pred)

plt.xlabel("X values")

plt.ylabel("Probability")

plt.title("Logistic Model")

plt.show()
```

OUTPUT:

Predicted Probabilities:

[0.11920292 0.26894142 0.5 0.73105858 0.88079708 0.95257413]



RESULT:

Thus, the Simple Linear Regression model was successfully implemented using the Least Squares Method.

Ex. No. : 04 **Date:**

Register No : 231701046 **Name: SAM SUNDAR**

A python program to implement single layer perceptron.

AIM:

To implement a Single Layer Perceptron model using Python and visualize the decision boundary using a graph.

REQUIREMENTS:

Jupiter Notebook with Python.

PROGRAM:

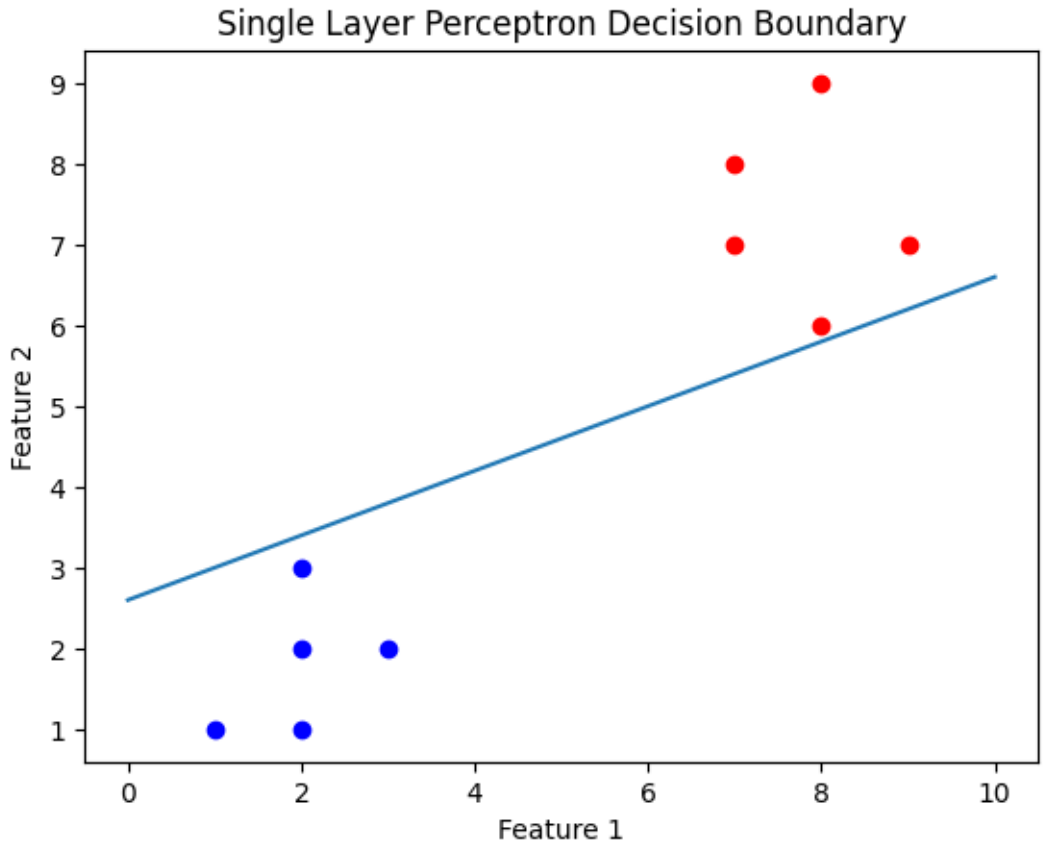
```
import numpy as np

import matplotlib.pyplot as plt

# Training data (two features)

X = np.array([[2,3], [1,1], [2,1], [3,2], [2,2], [7,8], [8,9], [7,7], [8,6], [9,7]])

# Target output (0 or 1)

y = np.array([0,0,0,0,0,1,1,1,1,1])

# Initialize weights and bias

weights = np.zeros(2)

bias = 0

lr = 0.1

epochs = 20

# Training the perceptron

for _ in range(epochs):

    for i in range(len(X)):
```

```
        net = np.dot(X[i], weights) + bias
        output = 1 if net > 0 else 0
        error = y[i] - output
        weights = weights + lr * error * X[i]
        bias = bias + lr * error
print("Weights:", weights)
print("Bias:", bias)
# Plot data points
for i in range(len(X)):
    if y[i] == 0:
        plt.scatter(X[i][0], X[i][1], color='blue')
    else:
        plt.scatter(X[i][0], X[i][1], color='red')
# Decision boundary line
x_values = np.linspace(0,10,100)
y_values = -(weights[0]*x_values + bias) / weights[1]
plt.plot(x_values, y_values)
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("Single Layer Perceptron Decision Boundary")
plt.show()
```

OUTPUT:

Weights: [-0.2 0.5]

Bias: -1.3



RESULT:

Thus, the Single Layer Perceptron model was successfully implemented using Python and the decision boundary was visualized using a graph.

Ex. No. : 05 **Date:**

Register No : 231701046 **Name: SAM SUNDAR**

A python program to implement multi layer perceptron with back propagation.

AIM:

To implement the **Multi Layer Perceptron (MLP) with Backpropagation** using Python.

REQUIREMENTS:

Jupiter Notebook with Python.

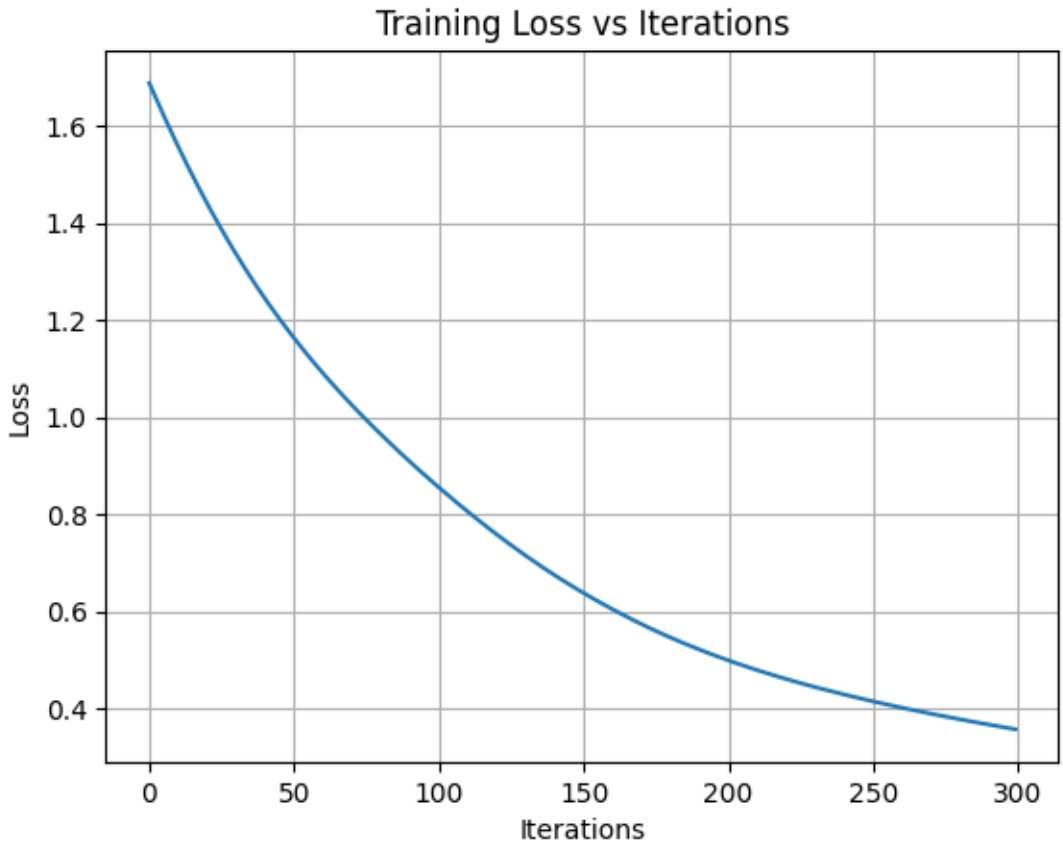
PROGRAM:

```
# Import libraries

import pandas as pdimport matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import LabelEncoder, StandardScaler

from sklearn.neural_network import MLPClassifier

from sklearn.metrics import accuracy_score


# Load dataset

data = pd.read_excel("iris_dataset.xlsx")

X = data.iloc[:,1:5]

y = data.iloc[:,5]

encoder = LabelEncoder()

y = encoder.fit_transform(y)
```

```
scaler = StandardScaler()
X = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# MLP Model
mlp = MLPClassifier(hidden_layer_sizes=(8,8),
                    activation='relu',
                    solver='adam',
                    max_iter=300,
                    random_state=1)
mlp.fit(X_train, y_train)
y_pred = mlp.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

plt.plot(mlp.loss_curve_)
plt.title("Training Loss vs Iterations")
plt.xlabel("Iterations")
plt.ylabel("Loss")
plt.grid(True)
plt.show()
```

OUTPUT:

Accuracy: 0.9333333333333333



RESULT:

Thus, the Simple Linear Regression model was successfully implemented using the Least Squares Method.

Ex. No. : 06 **Date:**

Register No : 231701046 **Name: Sam Sundar R**

A python program to do face recognition using SVM classifier

Aim:

To implement a python program to do face recognition using SVM classifier.

Requirements:

Jupyter Notebook with Python.

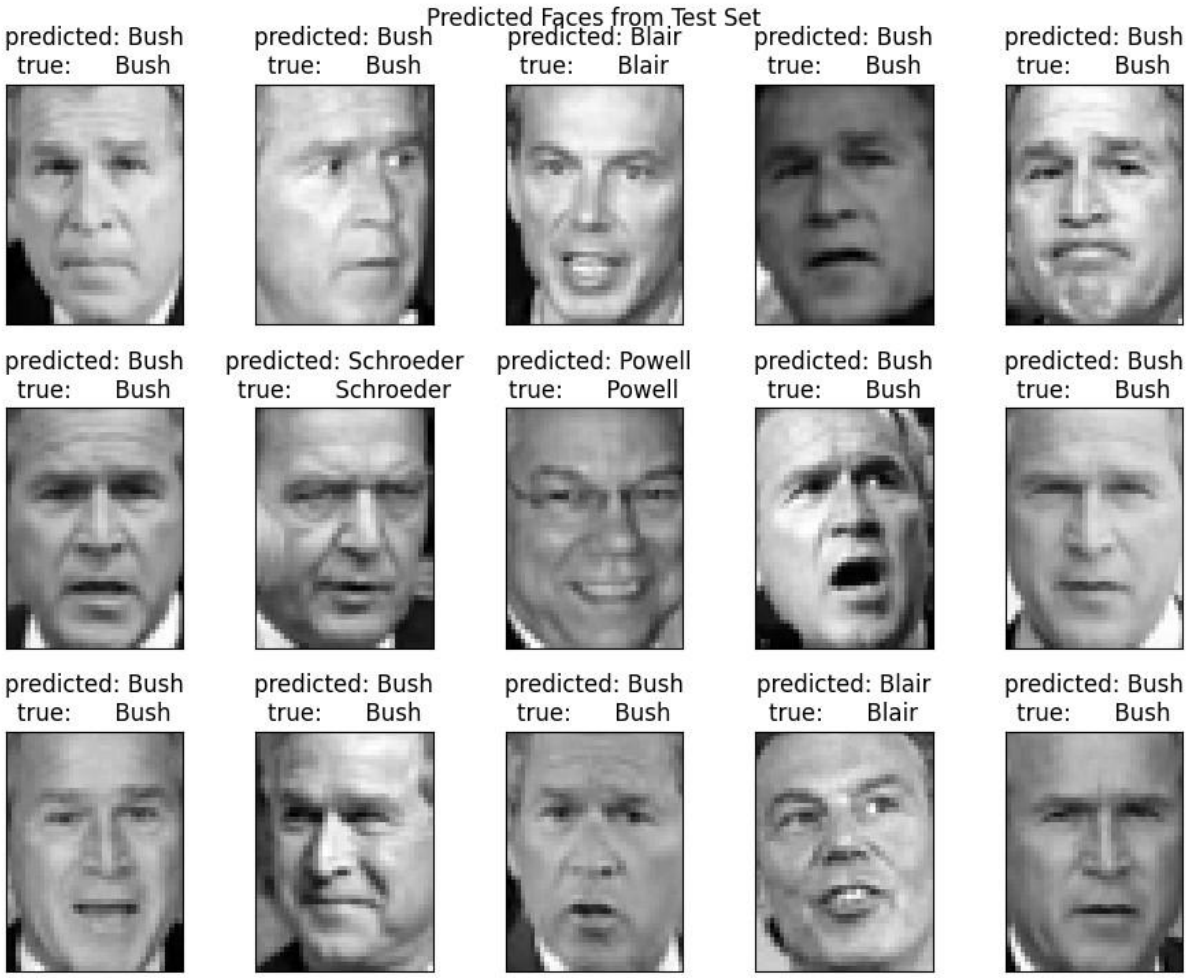
PROGRAM:

```
from sklearn.datasets import fetch_lfw_people from

sklearn.model_selection import train_test_split from

sklearn.preprocessing import StandardScaler from

sklearn.decomposition import PCA from sklearn.svm import SVC

from sklearn.metrics import classification_report, confusion_matrix

import matplotlib.pyplot as plt import numpy as np


print("Loading Labeled Faces in the Wild™ dataset (this may take a moment)...")


lfw_people = fetch_lfw_people(min_faces_per_person=70, resize=0.4)


# Introspect the data: image shape, target names
```

```
n_samples, h, w = lfw_people.images.shape

X = lfw_people.data n_features =

X.shape[1] y = lfw_people.target

target_names = lfw_people.target_names

n_classes = target_names.shape[0]


print(f"Total dataset size:") print(f"n_samples:

{n_samples}") print(f"n_features:

{n_features}") print(f"n_classes:

{n_classes}")


X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.25, random_state=42

)


print(f"Training set size: {X_train.shape[0]} samples") print(f"Testing

set size: {X_test.shape[0]} samples")


scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)


print("Extracting the top N 'eigenfaces' from the training set...")

n_components = 150
```

```
whiten=True,

pca = PCA(n_components=n_components,
svd_solver='randomized', random_state=42) pca.fit(X_train_scaled)

eigenfaces = pca.components_.reshape((n_components, h, w))

X_train_pca = pca.transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

print(f"Reduced dimensions from {n_features} to {n_components}")

print("Fitting the classifier to the training set...") clf =
SVC(kernel='rbf', C=1000, gamma=0.001, random_state=42)
clf.fit(X_train_pca, y_train) print("Classifier training complete.")

print("Predicting people's names on the test set...") y_pred
= clf.predict(X_test_pca)

print("\nClassification Report:") print(classification_report(y_test,
y_pred, target_names=target_names))

print("\nConfusion Matrix:") print(confusion_matrix(y_test,
y_pred, labels=range(n_classes)))
```

```
def plot_gallery(images, titles, h, w, n_row=3, n_col=4):
    plt.figure(figsize=(1.8 * n_col, 2.4 * n_row))
plt.subplots_adjust(bottom=0, left=.01, right=.99, top=.90, hspace=.35)     for
i in range(n_row * n_col):
        plt.subplot(n_row, n_col, i + 1)
plt.imshow(images[i].reshape((h, w)), cmap=plt.cm.gray)
        plt.title(titles[i], size=12)
plt.xticks(())        plt.yticks(())


def title(y_pred, y_test, target_names, i):
    pred_name = target_names[y_pred[i]].rsplit(' ', 1)[-1]
true_name = target_names[y_test[i]].rsplit(' ', 1)[-1]     return
'predicted: %s\ntrue:      %s' % (pred_name, true_name)


prediction_titles = [title(y_pred, y_test, target_names, i) for i in range(y_pred.shape[0])]


plot_gallery(X_test, prediction_titles, h, w, n_row=3, n_col=5)
plt.suptitle('Predicted Faces from Test Set')


plot_gallery(eigenfaces, [f'eigenface {i}' for i in range(eigenfaces.shape[0])], h, w)
plt.suptitle('Eigenfaces (First 12 components)')


plt.show()
```



Output:

Loading Labeled Faces in the Wild™ dataset (this may take a moment)...

Total dataset size:

n_samples: 1288

n_features: 1850 n_classes:

7

Training set size: 966 samples

Testing set size: 322 samples

Extracting the top N 'eigenfaces' from the training set...

Reduced dimensions from 1850 to 150

Fitting the classifier to the training set...

Classifier training complete.

Predicting people's names on the test set...

Classification Report:

	precision	recall	f1-score	support
Ariel Sharon	0.64	0.69	0.67	13
Colin Powell	0.75	0.85	0.80	60
Donald Rumsfeld	0.73	0.70	0.72	27
George W Bush	0.90	0.90	0.90	146
Gerhard Schroeder	0.80	0.80	0.80	25
Hugo Chavez	0.73	0.53	0.62	15
Tony Blair	0.94	0.83	0.88	36
accuracy			0.84	322
macro avg	0.78	0.76	0.77	322
weighted avg	0.84	0.84	0.83	322

Confusion Matrix:

```
[[ 9  1  2  1  0  0  0]
 [ 2 51  3  2  1  1  0]
 [ 3  1 19  3  0  1  0]
 [ 0  9  2 132  2  1  0]
 [ 0  1  0  3 20  0  1]
```

[0 3 0 2 1 8 1]

[0 2 0 3 1 0 30]] **Result:**

Thus, python program to do face recognition using SVM classifier has been successfully executed

Ex. No. : 07 **Date: 07-04-2026**

Register No : 231701046 **Name: Sam Sundar R**

A python program to implement decision tree

Aim:

To implement a Decision Tree classifier using Python and analyze classification on the given dataset.

Requirements:

Jupyter Notebook with Python.

PROGRAM:

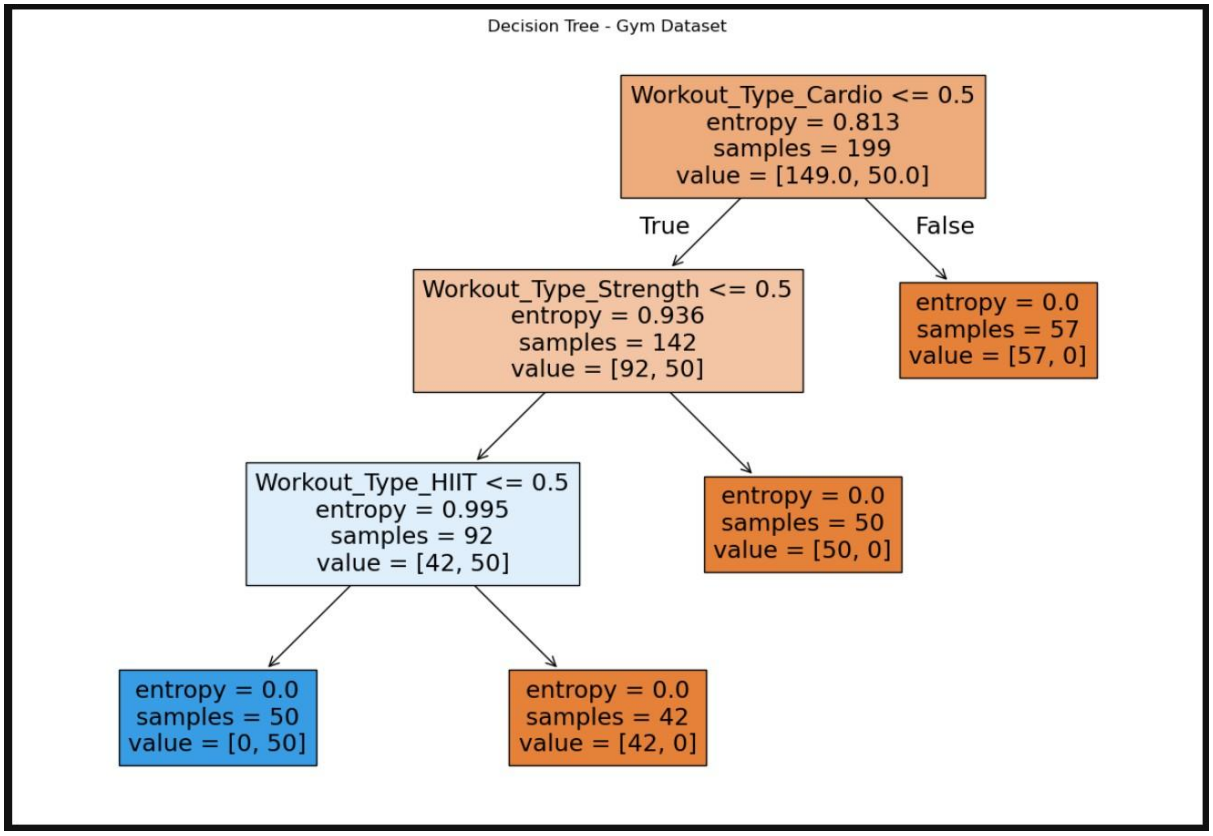
```
import pandas as pd import numpy as np from

sklearn.tree import DecisionTreeClassifier from

sklearn import tree from sklearn.metrics import

accuracy_score import matplotlib.pyplot as plt


# Load dataset df =

pd.read_csv('gym statistics.csv')


print("Dataset Preview:")

print(df.head())


# Convert categorical data to numeric

df_encoded = pd.get_dummies(df)
```

```
# Split features and target (last column as target)
X = df_encoded.iloc[:, :-1] y =
df_encoded.iloc[:, -1]

# Train Decision Tree model =
DecisionTreeClassifier(criterion='entropy')
model.fit(X, y)

# Predictions y_pred =
model.predict(X)

print("\nPredictions:") print(y_pred)

# Accuracy accuracy =
accuracy_score(y, y_pred)
print("\nAccuracy:", accuracy)

# Plot Decision Tree plt.figure(figsize=(15,10))
tree.plot_tree(model, feature_names=X.columns, filled=True)
plt.title("Decision Tree - Gym Dataset")
plt.show()
```

Output:

Feature shape: (199, 18)

Target shape: (199,)



Result:

The Decision Tree model was successfully trained and tested, achieving accurate classification on the dataset.

Ex. No. : 08 **Date: 14-04-2026**

Register No : 231701046 **Name: Sam Sundar R**

A python program to implement boosting

Aim:

To implement boosting using the AdaBoost algorithm on the given dataset for classification.

Requirements:

Jupyter Notebook with Python.

PROGRAM:

```
import pandas as pd import numpy as np from

sklearn.ensemble import AdaBoostClassifier from

sklearn.tree import DecisionTreeClassifier from

sklearn.metrics import accuracy_score from

sklearn.model_selection import train_test_split

import matplotlib.pyplot as plt


# Load dataset df = pd.read_csv('/mnt/data/gym

statistics(2).csv')


print("Dataset Preview:") print(df.head())


# Convert categorical data to numeric
df_encoded = pd.get_dummies(df)
```

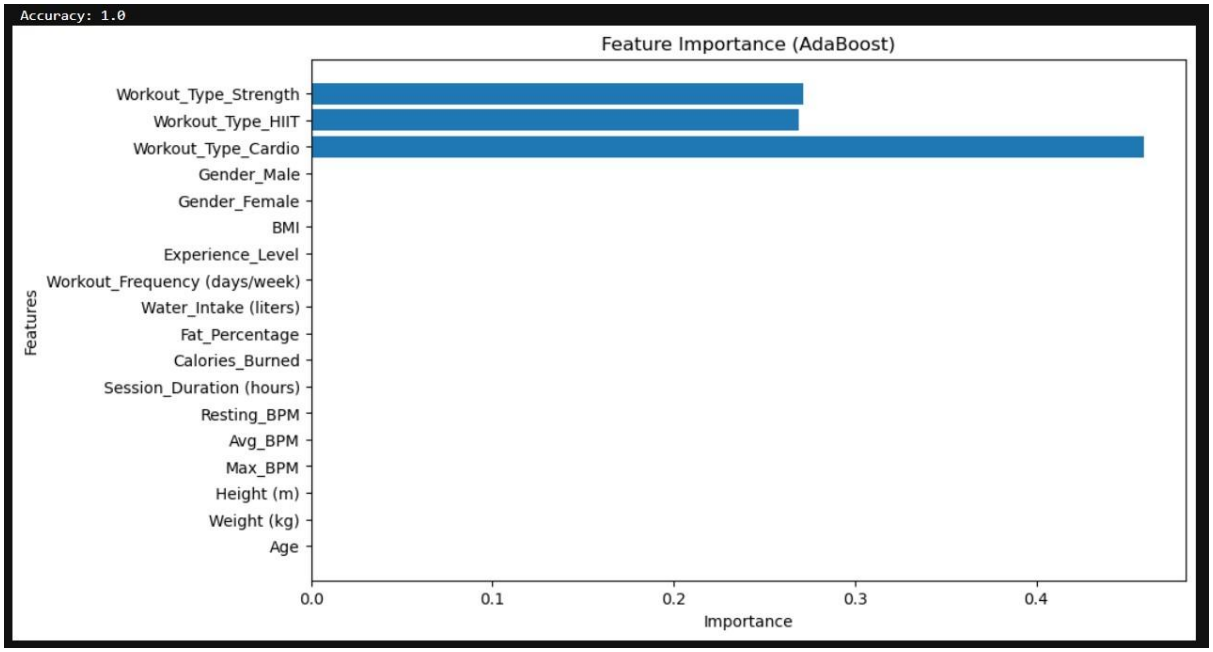
```
# Split features and target (last column as target)
X = df_encoded.iloc[:, :-1] y =
df_encoded.iloc[:, -1]

print("\nFeature shape:", X.shape) print("Target
shape:", y.shape)

# Train-test split (IMPORTANT)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Initialize AdaBoost with Decision Tree (stump) base_model
= DecisionTreeClassifier(max_depth=1)

model = AdaBoostClassifier(
estimator=base_model,
n_estimators=50,     learning_rate=1.0
)

# Train model model.fit(X_train,
y_train) # Predictions y_pred =
model.predict(X_test)

print("\nPredictions:") print(y_pred)
```

```
# Accuracy accuracy =

accuracy_score(y_test, y_pred)

print("\nAccuracy:", accuracy)


# Feature Importance Plot importances =

model.feature_importances_


plt.figure(figsize=(10,6))

plt.barh(X.columns, importances)

plt.title("Feature Importance (AdaBoost)")

plt.xlabel("Importance")

plt.ylabel("Features") plt.show()
```

Output:

Feature shape: (199, 18)

Target shape: (199,)



Result:

The AdaBoost model was successfully trained and achieved improved classification accuracy on the dataset.

Ex. No. : 09 **Date:**

Register No : 231701046 **Name: Sam Sundar R**

A python program to implement KNN and K-means

Aim:

To implement a python program that implement KNN and K-means

Requirements:

Jupyter Notebook with Python.

PROGRAM:

```
import numpy as np from

collections import Counter

import matplotlib.pyplot as plt


class KNearestNeighbors:

def __init__(self, k=3):

        self.k = k


    def fit(self, X, y):

self.X_train = X

self.y_train = y
```

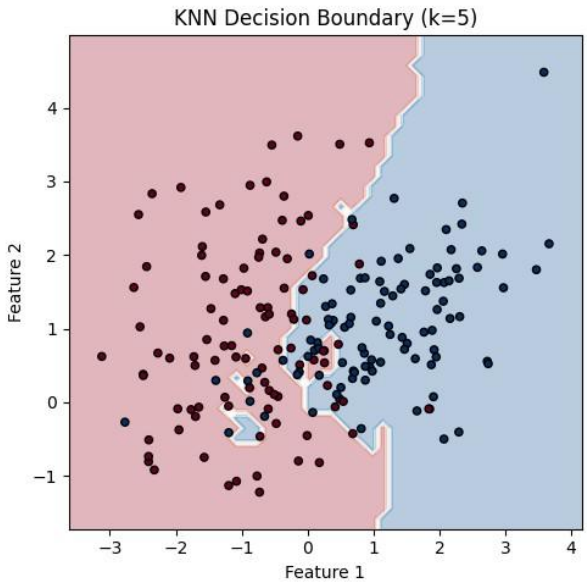
```
    def _euclidean_distance(self, x1, x2):
return np.sqrt(np.sum((x1 - x2)**2)) def
predict(self, X):

        predictions = [self._predict(x) for x in X]
return np.array(predictions)


    def _predict(self, x):
        distances = [self._euclidean_distance(x, x_train) for x_train in self.X_train]


        k_nearest_indices = np.argsort(distances)[:self.k]
k_nearest_labels = [self.y_train[i] for i in k_nearest_indices]


        most_common = Counter(k_nearest_labels).most_common(1)
return most_common[0][0]


class KMeans:     def __init__(self, n_clusters=3, max_iter=100,
random_state=None):
        self.n_clusters = n_clusters
self.max_iter = max_iter
self.random_state = random_state
self.centroids = None         self.labels
= None
```

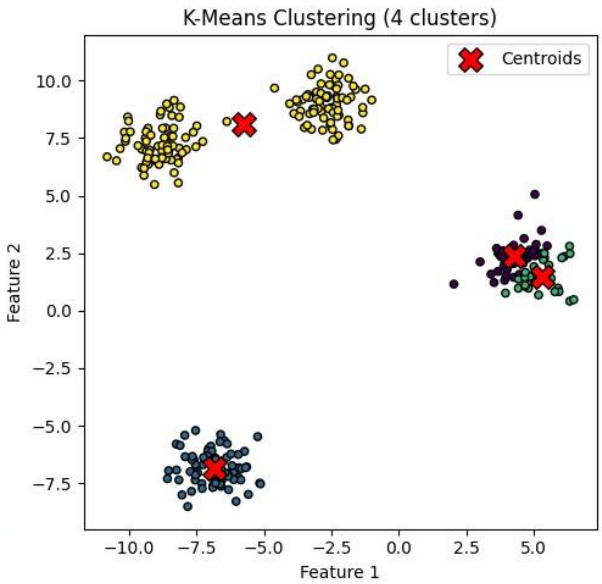
```
    def _initialize_centroids(self, X):
np.random.seed(self.random_state)        indices =
np.random.choice(X.shape[0], self.n_clusters, replace=False)     return
X[indices]


   def _assign_clusters(self, X, centroids):
        distances = np.array([
            [np.linalg.norm(x - centroid) for centroid in centroids]
for x in X
        ])
        return np.argmin(distances, axis=1)


    def _update_centroids(self, X, labels):
        new_centroids = np.zeros((self.n_clusters, X.shape[1]))
for i in range(self.n_clusters):
            points_in_cluster = X[labels == i]
if len(points_in_cluster) > 0:
                new_centroids[i] = np.mean(points_in_cluster, axis=0)
return new_centroids


    def fit(self, X):
        self.centroids = self._initialize_centroids(X)
```

```
        for _ in range(self.max_iter):
            self.labels = self._assign_clusters(X, self.centroids)
new_centroids = self._update_centroids(X, self.labels)

            if np.allclose(self.centroids, new_centroids):
                break        self.centroids =
    new_centroids

        return self.labels, self.centroids

    def predict(self, X):        if
self.centroids is None:
            raise RuntimeError("KMeans must be fitted before predicting.")
return self._assign_clusters(X, self.centroids)

if __name__ == "__main__":
    from sklearn.datasets import make_blobs, make_classification
from sklearn.model_selection import train_test_split     from
sklearn.metrics import accuracy_score

    print("\n--- K-Nearest Neighbors (KNN) Example ---")
    X_cls, y_cls = make_classification(n_samples=200, n_features=2, n_informative=2,
n_redundant=0, n_clusters_per_class=1, random_state=42)
```

```
X_train_cls, X_test_cls, y_train_cls, y_test_cls = train_test_split(X_cls, y_cls,
test_size=0.3, random_state=42)


knn = KNearestNeighbors(k=5)

knn.fit(X_train_cls, y_train_cls)     y_pred_knn

= knn.predict(X_test_cls)


accuracy_knn = accuracy_score(y_test_cls, y_pred_knn)

print(f"KNN Accuracy: {accuracy_knn * 100:.2f}%")


try:

    plt.figure(figsize=(10, 5))        plt.subplot(1, 2, 1)        x_min,

x_max = X_cls[:, 0].min() - 0.5, X_cls[:, 0].max() + 0.5        y_min,

y_max = X_cls[:, 1].min() - 0.5, X_cls[:, 1].max() + 0.5        xx, yy

= np.meshgrid(np.linspace(x_min, x_max, 50),

np.linspace(y_min, y_max, 50))

    Z_knn = knn.predict(np.c_[xx.ravel(), yy.ravel()])

    Z_knn = Z_knn.reshape(xx.shape)


    plt.contourf(xx, yy, Z_knn, alpha=0.3, cmap=plt.cm.RdBu)        plt.scatter(X_cls[:,

0], X_cls[:, 1], c=y_cls, s=20, edgecolor='k', cmap=plt.cm.RdBu)        plt.title('KNN

Decision Boundary (k=5)')        plt.xlabel('Feature 1')        plt.ylabel('Feature 2')


except ImportError:
```

```
        print("Matplotlib not installed. Skipping KNN plot.")

    print("\n--- K-Means Clustering Example ---")
X_km, y_true_km = make_blobs(n_samples=300, centers=4, cluster_std=0.8, random_state=42)  n_clusters_km = 4

    kmeans = KMeans(n_clusters=n_clusters_km, random_state=42)
    labels_km, centroids_km = kmeans.fit(X_km)

    print(f"K-Means performed with {n_clusters_km} clusters.")
print(f"Final Centroids:\n{centroids_km}")

try:
        plt.subplot(1, 2, 2)  plt.scatter(X_km[:, 0], X_km[:, 1], c=labels_km, s=20, cmap='viridis', edgecolor='k')  plt.scatter(centroids_km[:, 0], centroids_km[:, 1], marker='X', s=200, c='red', edgecolor='black', label='Centroids')
plt.title(f'K-Means Clustering ({n_clusters_km} clusters)')  plt.xlabel('Feature 1')
plt.ylabel('Feature 2')  plt.legend()  plt.tight_layout()  plt.show()

    except ImportError:
        print("Matplotlib not installed. Skipping K-Means plot.")
```





Output:

--- K-Nearest Neighbors (KNN) Example ---

KNN Accuracy: 80.00%

--- K-Means Clustering Example --- K-Means

performed with 4 clusters.

Final Centroids:

[[4.30482919 2.37577687]

 [-6.84180708 -6.84038791]

 [5.33339082 1.47653843]

 [-5.73523256 8.10177082]]

Result:

The python program has been successfully implemented

Ex. No. : **10** **Date:**

Register No : **231701046** **Name: Sam Sundar R**

A python program to implement dimentionality reduction - PCA

Aim:

To implement a python program that implement dimentionality reduction - PCA

Requirements:

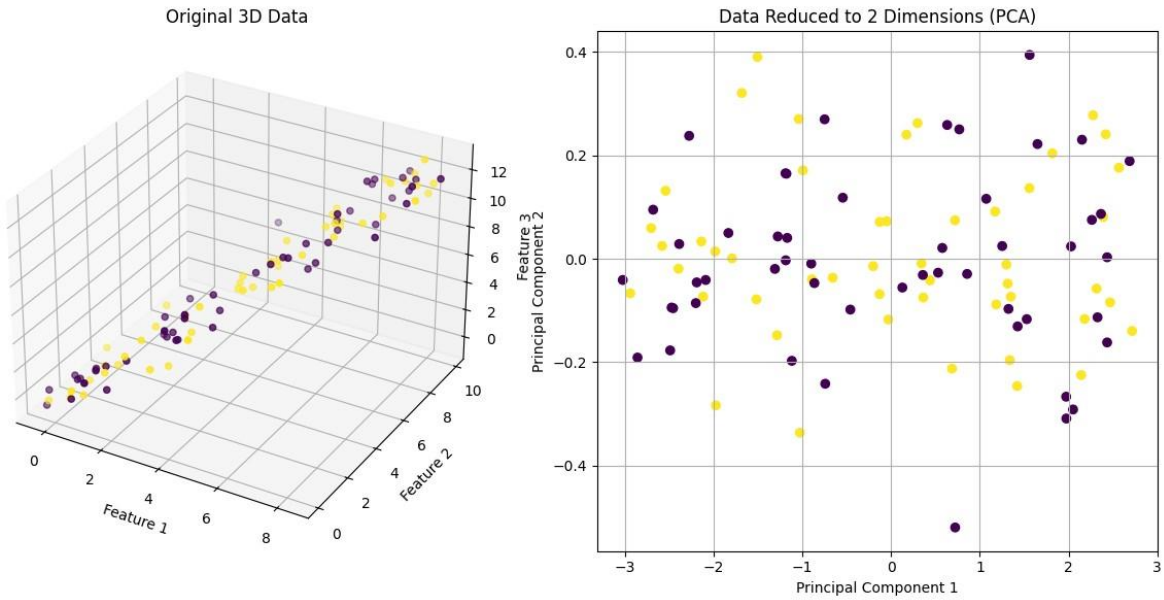
Jupyter Notebook with Python.

PROGRAM:

```
import numpy as np import matplotlib.pyplot as

plt from sklearn.decomposition import PCA from

sklearn.preprocessing import StandardScaler


print("--- Principal Component Analysis (PCA) Example ---")


np.random.seed(42)

X_3d = np.random.rand(100, 3) * 10

X_3d[:, 0] = X_3d[:, 1] * 0.8 + np.random.randn(100) * 0.5 X_3d[:,

2] = X_3d[:, 1] * 1.2 + np.random.randn(100) * 0.5 y_labels =

np.random.randint(0, 2, 100)


print(f"Original data shape: {X_3d.shape}")
```

```
scaler = StandardScaler()
X_3d_scaled = scaler.fit_transform(X_3d)

n_components = 2 pca =
PCA(n_components=n_components)
X_2d = pca.fit_transform(X_3d_scaled)

print(f"Reduced data shape: {X_2d.shape}") print(f"Explained variance ratio by each
component: {pca.explained_variance_ratio_}") print(f"Total explained variance:
{pca.explained_variance_ratio_.sum():.2f}")

fig = plt.figure(figsize=(12, 6)) ax = fig.add_subplot(121,
projection='3d') ax.scatter(X_3d[:, 0], X_3d[:, 1], X_3d[:, 2],
c=y_labels, cmap='viridis') ax.set_title('Original 3D Data')
ax.set_xlabel('Feature 1') ax.set_ylabel('Feature 2')
ax.set_zlabel('Feature 3')

ax_2d = fig.add_subplot(122) ax_2d.scatter(X_2d[:, 0], X_2d[:, 1],
c=y_labels, cmap='viridis') ax_2d.set_title(f'Data Reduced to
{n_components} Dimensions (PCA)') ax_2d.set_xlabel('Principal
Component 1') ax_2d.set_ylabel('Principal Component 2')

ax_2d.grid(True)
```

plt.tight_layout() plt.show()



Output:

--- Principal Component Analysis (PCA) Example ---

Original data shape: (100, 3)

Reduced data shape: (100, 2)

Explained variance ratio by each component: [0.98827418 0.00918499] Total

explained variance: 1.00

Result:

The python program has been successfully implemented

Id	SepalLengt	SepalWidtl	PetalLengt	PetalWidtl	Species
1	5.1	3.5	1.4	0.2	Iris-setosa
2	4.9	3	1.4	0.2	Iris-setosa
3	4.7	3.2	1.3	0.2	Iris-setosa
4	4.6	3.1	1.5	0.2	Iris-setosa
5	5	3.6	1.4	0.2	Iris-setosa
6	5.4	3.9	1.7	0.4	Iris-setosa
7	4.6	3.4	1.4	0.3	Iris-setosa
8	5	3.4	1.5	0.2	Iris-setosa
9	4.4	2.9	1.4	0.2	Iris-setosa
10	4.9	3.1	1.5	0.1	Iris-setosa
11	5.4	3.7	1.5	0.2	Iris-setosa
12	4.8	3.4	1.6	0.2	Iris-setosa
13	4.8	3	1.4	0.1	Iris-setosa
14	4.3	3	1.1	0.1	Iris-setosa
15	5.8	4	1.2	0.2	Iris-setosa
16	5.7	4.4	1.5	0.4	Iris-setosa
17	5.4	3.9	1.3	0.4	Iris-setosa
18	5.1	3.5	1.4	0.3	Iris-setosa
19	5.7	3.8	1.7	0.3	Iris-setosa
20	5.1	3.8	1.5	0.3	Iris-setosa
21	5.4	3.4	1.7	0.2	Iris-setosa
22	5.1	3.7	1.5	0.4	Iris-setosa
23	4.6	3.6	1	0.2	Iris-setosa
24	5.1	3.3	1.7	0.5	Iris-setosa
25	4.8	3.4	1.9	0.2	Iris-setosa
26	5	3	1.6	0.2	Iris-setosa
27	5	3.4	1.6	0.4	Iris-setosa
28	5.2	3.5	1.5	0.2	Iris-setosa
29	5.2	3.4	1.4	0.2	Iris-setosa
30	4.7	3.2	1.6	0.2	Iris-setosa
31	4.8	3.1	1.6	0.2	Iris-setosa
32	5.4	3.4	1.5	0.4	Iris-setosa
33	5.2	4.1	1.5	0.1	Iris-setosa
34	5.5	4.2	1.4	0.2	Iris-setosa
35	4.9	3.1	1.5	0.1	Iris-setosa
36	5	3.2	1.2	0.2	Iris-setosa
37	5.5	3.5	1.3	0.2	Iris-setosa
38	4.9	3.1	1.5	0.1	Iris-setosa
39	4.4	3	1.3	0.2	Iris-setosa
40	5.1	3.4	1.5	0.2	Iris-setosa
41	5	3.5	1.3	0.3	Iris-setosa
42	4.5	2.3	1.3	0.3	Iris-setosa
43	4.4	3.2	1.3	0.2	Iris-setosa
44	5	3.5	1.6	0.6	Iris-setosa
45	5.1	3.8	1.9	0.4	Iris-setosa
46	4.8	3	1.4	0.3	Iris-setosa

47	5.1	3.8	1.6	0.2	Iris-setosa
48	4.6	3.2	1.4	0.2	Iris-setosa
49	5.3	3.7	1.5	0.2	Iris-setosa
50	5	3.3	1.4	0.2	Iris-setosa
51	7	3.2	4.7	1.4	Iris-versicolor
52	6.4	3.2	4.5	1.5	Iris-versicolor
53	6.9	3.1	4.9	1.5	Iris-versicolor
54	5.5	2.3	4	1.3	Iris-versicolor
55	6.5	2.8	4.6	1.5	Iris-versicolor
56	5.7	2.8	4.5	1.3	Iris-versicolor
57	6.3	3.3	4.7	1.6	Iris-versicolor
58	4.9	2.4	3.3	1	Iris-versicolor
59	6.6	2.9	4.6	1.3	Iris-versicolor
60	5.2	2.7	3.9	1.4	Iris-versicolor
61	5	2	3.5	1	Iris-versicolor
62	5.9	3	4.2	1.5	Iris-versicolor
63	6	2.2	4	1	Iris-versicolor
64	6.1	2.9	4.7	1.4	Iris-versicolor
65	5.6	2.9	3.6	1.3	Iris-versicolor
66	6.7	3.1	4.4	1.4	Iris-versicolor
67	5.6	3	4.5	1.5	Iris-versicolor
68	5.8	2.7	4.1	1	Iris-versicolor
69	6.2	2.2	4.5	1.5	Iris-versicolor
70	5.6	2.5	3.9	1.1	Iris-versicolor
71	5.9	3.2	4.8	1.8	Iris-versicolor
72	6.1	2.8	4	1.3	Iris-versicolor
73	6.3	2.5	4.9	1.5	Iris-versicolor
74	6.1	2.8	4.7	1.2	Iris-versicolor
75	6.4	2.9	4.3	1.3	Iris-versicolor
76	6.6	3	4.4	1.4	Iris-versicolor
77	6.8	2.8	4.8	1.4	Iris-versicolor
78	6.7	3	5	1.7	Iris-versicolor
79	6	2.9	4.5	1.5	Iris-versicolor
80	5.7	2.6	3.5	1	Iris-versicolor
81	5.5	2.4	3.8	1.1	Iris-versicolor
82	5.5	2.4	3.7	1	Iris-versicolor
83	5.8	2.7	3.9	1.2	Iris-versicolor
84	6	2.7	5.1	1.6	Iris-versicolor
85	5.4	3	4.5	1.5	Iris-versicolor
86	6	3.4	4.5	1.6	Iris-versicolor
87	6.7	3.1	4.7	1.5	Iris-versicolor
88	6.3	2.3	4.4	1.3	Iris-versicolor
89	5.6	3	4.1	1.3	Iris-versicolor
90	5.5	2.5	4	1.3	Iris-versicolor
91	5.5	2.6	4.4	1.2	Iris-versicolor
92	6.1	3	4.6	1.4	Iris-versicolor
93	5.8	2.6	4	1.2	Iris-versicolor

94	5	2.3	3.3	1	Iris-versicolor
95	5.6	2.7	4.2	1.3	Iris-versicolor
96	5.7	3	4.2	1.2	Iris-versicolor
97	5.7	2.9	4.2	1.3	Iris-versicolor
98	6.2	2.9	4.3	1.3	Iris-versicolor
99	5.1	2.5	3	1.1	Iris-versicolor
100	5.7	2.8	4.1	1.3	Iris-versicolor
101	6.3	3.3	6	2.5	Iris-virginica
102	5.8	2.7	5.1	1.9	Iris-virginica
103	7.1	3	5.9	2.1	Iris-virginica
104	6.3	2.9	5.6	1.8	Iris-virginica
105	6.5	3	5.8	2.2	Iris-virginica
106	7.6	3	6.6	2.1	Iris-virginica
107	4.9	2.5	4.5	1.7	Iris-virginica
108	7.3	2.9	6.3	1.8	Iris-virginica
109	6.7	2.5	5.8	1.8	Iris-virginica
110	7.2	3.6	6.1	2.5	Iris-virginica
111	6.5	3.2	5.1	2	Iris-virginica
112	6.4	2.7	5.3	1.9	Iris-virginica
113	6.8	3	5.5	2.1	Iris-virginica
114	5.7	2.5	5	2	Iris-virginica
115	5.8	2.8	5.1	2.4	Iris-virginica
116	6.4	3.2	5.3	2.3	Iris-virginica
117	6.5	3	5.5	1.8	Iris-virginica
118	7.7	3.8	6.7	2.2	Iris-virginica
119	7.7	2.6	6.9	2.3	Iris-virginica
120	6	2.2	5	1.5	Iris-virginica
121	6.9	3.2	5.7	2.3	Iris-virginica
122	5.6	2.8	4.9	2	Iris-virginica
123	7.7	2.8	6.7	2	Iris-virginica
124	6.3	2.7	4.9	1.8	Iris-virginica
125	6.7	3.3	5.7	2.1	Iris-virginica
126	7.2	3.2	6	1.8	Iris-virginica
127	6.2	2.8	4.8	1.8	Iris-virginica
128	6.1	3	4.9	1.8	Iris-virginica
129	6.4	2.8	5.6	2.1	Iris-virginica
130	7.2	3	5.8	1.6	Iris-virginica
131	7.4	2.8	6.1	1.9	Iris-virginica
132	7.9	3.8	6.4	2	Iris-virginica
133	6.4	2.8	5.6	2.2	Iris-virginica
134	6.3	2.8	5.1	1.5	Iris-virginica
135	6.1	2.6	5.6	1.4	Iris-virginica
136	7.7	3	6.1	2.3	Iris-virginica
137	6.3	3.4	5.6	2.4	Iris-virginica
138	6.4	3.1	5.5	1.8	Iris-virginica
139	6	3	4.8	1.8	Iris-virginica
140	6.9	3.1	5.4	2.1	Iris-virginica

141	6.7	3.1	5.6	2.4	Iris-virginica
142	6.9	3.1	5.1	2.3	Iris-virginica
143	5.8	2.7	5.1	1.9	Iris-virginica
144	6.8	3.2	5.9	2.3	Iris-virginica
145	6.7	3.3	5.7	2.5	Iris-virginica
146	6.7	3	5.2	2.3	Iris-virginica
147	6.3	2.5	5	1.9	Iris-virginica
148	6.5	3	5.2	2	Iris-virginica
149	6.2	3.4	5.4	2.3	Iris-virginica
150	5.9	3	5.1	1.8	Iris-virginica